Variant 5: Prefix Expression Workbench

Team ID: ***18***

Shaymaa Mohamed Reda *20230292*

Sondos Mohamed Abd El-Sattar *20230268*

Fatima Abdullah Swelem *20230401*

Mohamed Anany Abd El-Latif *20230494*

Mohamed Mostafa Mahmoud *20230507*

Mohamed Waleed Mohamed *20230516*

1. Grammar Description (EBNF)

The workbench follows an **LL(1)** grammar designed for recursive-descent parsing, ensuring nested prefix forms are handled with absolute structural integrity.

Program -> StmtSeq
StmtSeq -> Expression StmtSeq | ε
Expression -> Atom | List
Atom -> number | id | boolean
List -> '(' Form ')'
Form -> operator ArgList | 'let' id Expression
ArgList -> Expression ArgList | ε
operator -> '+' | '-' | '' | '/' | 'and' | 'or' | 'not' | '>' | '<' | '='*

2. Token Specification

The scanner uses a **rule-based** approach to recognize tokens, distinguishing between identifiers and reserved keywords using a static mapping.

Token Type	Lexeme Pattern	Description / Literal
LPAREN / RPAREN	(,)	Expression delimiters
PLUS / MINUS	+ , -	Arithmetic operators
STAR / SLASH	* , /	Arithmetic operators
GREATER / LESS / EQUAL	> , < , =	Comparison operators
LET / AND / OR / NOT	let, and, or, not.	Language keywords
TRUE / FALSE	true , false	Boolean literals
NUMBER	[0-9]+	Integer literals
IDENTIFIER	[a-zA-Z][a-zA-Z0-9]*	Variable names

3. Java Source Code

The implementation is modularized into six core files:

- **`App.java`**: Acts as the main entry point, containing the automated batch testing suite and the interactive REPL (Read-Evaluate-Print Loop) mode.
- **`Scanner.java`**: A handwritten, rule-based lexical analyzer that converts raw source text into a list of tokens.
- **`Parser.java`**: A handwritten recursive-descent parser that implements the prefix expression grammar.
- **`Node.java`**: Defines the Abstract Syntax Tree (AST) hierarchy through an abstract base class and specific node subclasses.
- **`Token.java`**: A data structure used to store information about identified lexemes, their types, and literal values.
- **`TokenType.java`**: An enumeration defining the complete set of valid tokens recognized by the language.

4. Valid and Invalid Test Cases

The project includes a robust suite of tests implemented in `App.java` to verify both system stability and error reporting.

Category	Sample Input	Expected Behavior / Proof of Correctness
Variadic Args	(+ 10 20 30 40)	**Success:** Parsed as an ApplicationNode with 4 children.
Nested Expression	(* (+ 1 2) (- 10 5))	**Success:** Confirms recursive structure handles operators as arguments.
Variable Binding	(let x 50) (* x 2)	**Success:** Correctly identifies the LET keyword and builds a LetNode for identifier x.
Boolean Logic	(and true (> 10 5))	**Success:** Demonstrates integration of logical operators and arithmetic comparisons.
Malformed Input	(+ 5 (* 2 3)	**Caught:** The parser identifies the missing RPAREN and triggers a high-visibility error block: "Expect ')' after arguments."
Invalid Character	(let price $100)	**Caught:** The scanner identifies $ as an illegal character and triggers a lexical error.
Very Long Expression	(+ 10 20 30 40 50 60 70 80 90 100)	**Success:** Confirms the while loop logic for argument lists handles long variadic sequences.
Deeply Nested Expression	(+ 1 (+ 2 (+ 3 ...)))	**Caught/Success:** Verifies the recursive stability of the parser; identifies errors even at extreme depths (e.g., "Expected expression").
Complex Mixed Logic	(and true (> (* 2 5) (+ 3 4)))	**Success:** Confirms deep integration of arithmetic and logic within comparison forms.

Empty List	()	**Caught:** The parser identifies that a list was opened but closed immediately without providing a required operator. **Error:**"Expected an operator after '(', but found ')'."
Keyword Reservation	(let and 10)	**Caught:** The parser prevents using reserved keywords (e.g., and) as variable identifiers.
Keyword Reservation 2	(let let 10)	**Caught:** Specifically blocks the use of the let keyword as an identifier name.
Overflow Error	(+ 999999999999 1)	**Caught:** The scanner detects numeric literals exceeding the 32-bit integer limit (2,147,483,647).
Division by Zero	(/ 10 0)	**Success (Parsed):** The code builds a valid AST because the syntax is correct; literal division by zero is a semantic error outside the core front-end parsing scope.

5. AST Output Examples

The system produces a visual representation of the AST using the `prettyPrint` method. Below is the standard output for the internal batch testing suite:

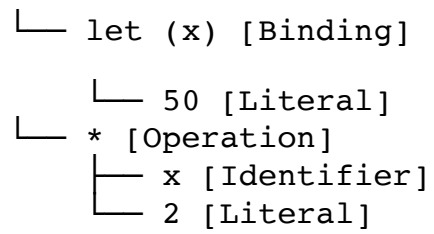
- **Variadic Arguments Test (+ 10 20 30 40)**

```
└── + [Operation]
    ├── 10 [Literal]
    ├── 20 [Literal]
    ├── 30 [Literal]
    └── 40 [Literal]
```

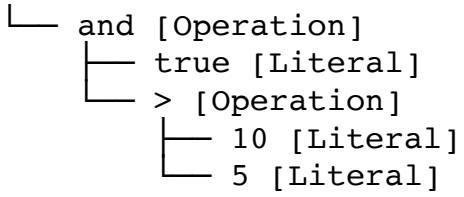
- **Nested Expression Test (* (+ 1 2) (- 10 5))**

```
└── * [Operation]
    ├── + [Operation]
    │   ├── 1 [Literal]
    │   └── 2 [Literal]
    └── - [Operation]
        ├── 10 [Literal]
        └── 5 [Literal]
```

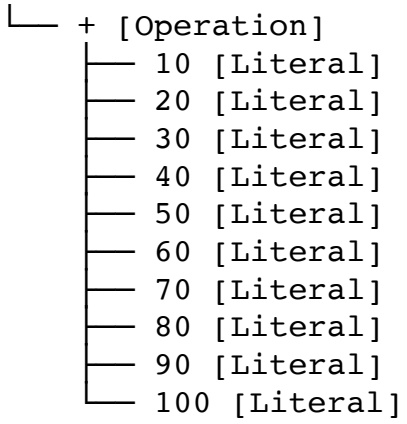
- **Variable Binding Test (let x 50) (* x 2)**



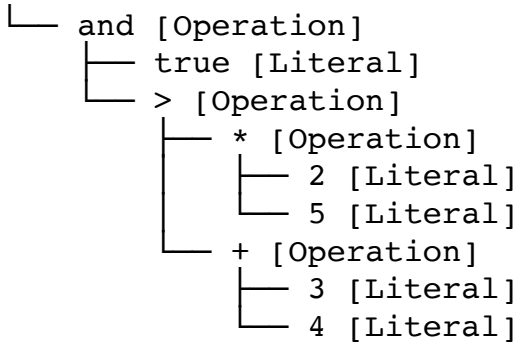
- **Boolean Logic Test (and true (> 10 5))**



- **Very Long Variadic Addition (+ 10 20 30 40 50 60 70 80 90 100)**



- **Complex Mixed Logic (and true (> (* 2 5) (+ 3 4)))**

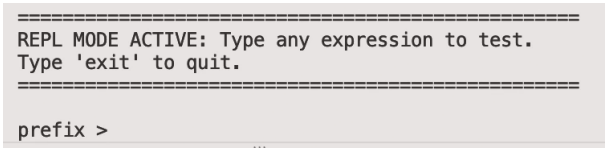


6. Short Design Report

- **Scanner Independence**: The `Scanner` class operates as a distinct module, processing raw source strings into a `List<Token>` before parsing begins. This separation follows professional standards by decoupling lexical analysis from syntax analysis, enabling independent testing and clear error reporting.
- **Recursive Structure**: To manage the LISP-like nested syntax, the parser uses a Top-Down Recursive Descent strategy. The implementation relies on a recursive relationship where `expression()` calls `list()`, which in turn recursively calls `expression()` to handle variadic arguments, allowing the workbench to process deep nesting without structural failure.
- **AST Hierarchy & Polymorphism**: The tree is built using a polymorphic `Node` base class. Subclasses such as `LiteralNode`, `IdNode`, `ApplicationNode`, and `LetNode` handle diverse data types through a unified interface, with each implementing its own `prettyPrint` logic for visual terminal output.
- **Terminal Readability & Indentation**: To ensure demo readiness, the system uses a standardized `INDENT` constant. This indentation is passed through recursive `prettyPrint` calls so that deeply nested trees remain visually aligned for the user.
- **Front-End Robustness & Scope**: The project prioritizes high-integrity syntax and lexical analysis.
    - **High-Visibility Diagnostics**: Both lexical and syntax errors trigger high-visibility blocks with exclamation-point borders (!!!) to ensure they are noticed during live use.
    - **Semantic vs. Syntax Scope**: While the language parses division, literal division by zero (e.g., `(/ 10 0)`) is classified as a Semantic Error. Such cases are out of scope for this workbench, which focuses on structural parsing rather than final evaluation.

7. Demo Description

- **Interactive REPL Mode**: The workbench provides a live environment for Construction construction construction exploring prefix expression parsing and AST construction.



- **Launch Phase (System Audit)**: Upon execution, the system runs an automated batch testing suite. This includes stress tests for deep recursive nesting (e.g., `(+ 1 (+ 2 (+ 3...)))`) to prove the stability of the recursive-descent logic.
- **Interaction Phase**: After the audit, the program enters a persistent loop where users can input custom expressions, including variable bindings like `(let x 50)` or logical comparisons.
- **Indented Feedback Loop**: For every valid input, the system displays two levels of feedback:
    1. **The Token Stream**: A table showing `TokenType`, lexemes, and interpreted literals.
    2. **The Visual AST**: A graphical tree representation of the expression structure.
- **Readability**: All feedback is indented by four spaces to keep compiler output distinct from user input and terminal labels.
- **High-Visibility Error Handling**: The system treats lexical errors (e.g., unexpected characters or integer overflow) and syntax errors as high-priority events. These trigger diagnostic blocks wrapped in exclamation-point borders (!!!), identifying exactly where input was blocked while keeping the REPL active.

```
================================================
LABEL:  Complex Mixed Logic
SOURCE: (and true (> (* 2 5) (+ 3 4)))
================================================

    [1] SCANNER (Token Stream):
    TYPE                | LEXEME          | LITERAL
    ------------------------------------------------
    LPAREN              | (               | null
    AND                 | and             | null
    TRUE                | true            | true
    LPAREN              | (               | null
    GREATER             | >               | null
    LPAREN              | (               | null
    STAR                | *               | null
    NUMBER              | 2               | 2
    NUMBER              | 5               | 5
    RPAREN              | )               | null
    LPAREN              | (               | null
    PLUS                | +               | null
    NUMBER              | 3               | 3
    NUMBER              | 4               | 4
    RPAREN              | )               | null
    RPAREN              | )               | null
    RPAREN              | )               | null

    [2] PARSER (Visual AST Structure):
    ------------------------------------------------
    └── and [Operation]
        ├── true [Literal]
        └── > [Operation]
            ├── * [Operation]
            │   ├── 2 [Literal]
            │   └── 5 [Literal]
            └── + [Operation]
                ├── 3 [Literal]
                └── 4 [Literal]
```

```
================================================
LABEL:  Very Long Variadic Addition
SOURCE: (+ 10 20 30 40 50 60 70 80 90 100)
================================================

    [1] SCANNER (Token Stream):
    TYPE                | LEXEME          | LITERAL
    ------------------------------------------------
    LPAREN              | (               | null
    PLUS                | +               | null
    NUMBER              | 10              | 10
    NUMBER              | 20              | 20
    NUMBER              | 30              | 30
    NUMBER              | 40              | 40
    NUMBER              | 50              | 50
    NUMBER              | 60              | 60
    NUMBER              | 70              | 70
    NUMBER              | 80              | 80
    NUMBER              | 90              | 90
    NUMBER              | 100             | 100
    RPAREN              | )               | null

    [2] PARSER (Visual AST Structure):
    ------------------------------------------------
    └── + [Operation]
        ├── 10 [Literal]
        ├── 20 [Literal]
        ├── 30 [Literal]
        ├── 40 [Literal]
        ├── 50 [Literal]
        ├── 60 [Literal]
        ├── 70 [Literal]
        ├── 80 [Literal]
        ├── 90 [Literal]
        └── 100 [Literal]
```

```
================================================
LABEL:  Malformed Input Test
SOURCE: (+ 5 (* 2 3)
================================================

    [1] SCANNER (Token Stream):
    TYPE                | LEXEME          | LITERAL
    ------------------------------------------------
    LPAREN              | (               | null
    PLUS                | +               | null
    NUMBER              | 5               | 5
    LPAREN              | (               | null
    STAR                | *               | null
    NUMBER              | 2               | 2
    NUMBER              | 3               | 3
    RPAREN              | )               | null

    [2] PARSER (Visual AST Structure):
    ------------------------------------------------
[Line 1] Error at '': Expect ')' after arguments.

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!! FRONT-END ERROR TRIGGERED !!!
! > null
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

```
================================================
LABEL:  Empty List Test
SOURCE: ()
================================================

    [1] SCANNER (Token Stream):
    TYPE                | LEXEME          | LITERAL
    ------------------------------------------------
    LPAREN              | (               | null
    RPAREN              | )               | null

    [2] PARSER (Visual AST Structure):
    ------------------------------------------------

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
>>> FRONT-END ERROR: [Line 1] Expected an operator after '(', but found ')'.
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

```
================================================
LABEL:  Variable Binding Test
SOURCE: (let x 50) (* x 2)
================================================

    [1] SCANNER (Token Stream):
    TYPE                | LEXEME          | LITERAL
    ------------------------------------------------
    LPAREN              | (               | null
    LET                 | let             | null
    IDENTIFIER          | x               | null
    NUMBER              | 50              | 50
    RPAREN              | )               | null
    LPAREN              | (               | null
    STAR                | *               | null
    IDENTIFIER          | x               | null
    NUMBER              | 2               | 2
    RPAREN              | )               | null

    [2] PARSER (Visual AST Structure):
    ------------------------------------------------
    └── let (x) [Binding]
        └── 50 [Literal]
    └── * [Operation]
        ├── x [Identifier]
        └── 2 [Literal]
```

```
================================================
LABEL:  Invalid Character Test
SOURCE: (let price $100)
================================================

    [1] SCANNER (Token Stream):
    TYPE                | LEXEME          | LITERAL
    ------------------------------------------------

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!! FRONT-END ERROR TRIGGERED !!!
! > Lexical Error at Line 1: Unexpected character: $
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```